

Migración de React / TypeScript a AVFenix Types

Guía técnica para la modernización de interfaces con Persistencia de Esquema Dual

1. Introducción y Motivación Arquitectónica

AVFenix Types ofrece un paradigma de desarrollo diseñado para resolver las limitaciones estructurales de frameworks tradicionales como React y TypeScript. Mientras que TypeScript realiza eliminación de tipos (*Type Erasure*) durante la transpilación, el compilador **AFXC** de AVFenix implementa **Persistencia de Esquema Dual**, generando simultáneamente manifiestos JSON (`.schema.json`) para backend/CMS y bundles JavaScript (`.js`) optimizados con Virtual DOM y *AST Static Hoisting* para el cliente en General.JS.

Beneficio Clave de la Migración: Un único archivo `.avf` define los tipos de datos, la interfaz de usuario con decoradores `@ui`, las reglas de negocio con `@validate` y las relaciones de base de datos relacionales con `@link`, eliminando la duplicidad de interfaces entre cliente y servidor.

2. Tabla de Equivalencias de Conceptos

Concepto en React / TS	Equivalente en AVFenix Types	Descripción & Ventaja
tsx / jsx	.avf (Component & Schema)	Archivo unificado con definición de tipos, estado y sintaxis JSX.
useState(init)	this.state / this.setState()	Reactividad local profunda gestionada por Proxies en reactive.general.js.
useEffect(fn, deps)	onMount() / onDestroy()	Hooks de ciclo de vida explícitos y transparentes sin re-renders fantasma.
Redux / Context API	@avfenix/store	Estado global reactivo con Time-Travel debugging y Signals ultra-ligeros.
React Router	routing.general.js	Mapeo SPA con guards de seguridad y precarga de datos.
Axios / Zod / Fetch	@avfenix/client	Cliente HTTP con validación automática pre-envío según .schema.json.
Next.js SSR	@avfenix/ssr	Renderizado en servidor y rehidratación en el cliente (AVFenixHydrate).
Jest / Testing Library	@avfenix/test-utils	Montaje aislado y simulación de eventos sobre el Virtual DOM.

Migración de React / TypeScript a AVFenix Types

Guía técnica para la modernización de interfaces con Persistencia de Esquema Dual

Paso 1: Inicialización del Ecosistema: Ejecuta `npx avfenix create mi-proyecto` o instala las dependencias de General.JS (`gnrl.js`, `reactive.general.js`) en tu proyecto existente.

Paso 2: Conversión de Interfaces a Esquemas .avf: Reemplaza tus archivos `.ts` de interfaces por esquemas `schema` en archivos `.avf`, agregando decoradores `@ui` y `@validate`.

Paso 3: Conversión de Componentes TSX a .avf: Convierte tus componentes funcionales o clases de React a la sintaxis `component Nombre { state = {}; template(state) { return (...) } }`.

Paso 4: Migración de Estado y Rutas: Sustituye Context API / Redux por `@avfenix/store` y configura tus rutas con `routing.general.js`.

Paso 5: Auditoría A11y y Pruebas: Ejecuta `npx avfenix check:a11y` para asegurar el cumplimiento WCAG 2.1 AA y valida con `@avfenix/test-utils`.

4. Ejemplo Práctico Comparativo

```
// Código Original en React + TypeScript
(ContadorProducto.tsx)
import React, { useState } from 'react';
interface Props { initial: number; name:
string; }
export const ContadorProducto:
React.FC<Props> = ({ initial, name }) => {
  const [count, setCount] =
    useState(initial);
  return (
    <div className='card'>
      <h3>{name}</h3>
      <button onClick={() => setCount(count +
1)}>Cantidad: {count}</button>
    </div>
  );
};
```

```
// Código Migrado a AVFenix Types
(ContadorProducto.avf)
export component ContadorProducto {
  state = { count: 0 };
  onMount() {
    this.setState({ count: this.props.initial
|| 0 });
  }
  template(state) {
    return (
      <div class="card">
        <h3>{this.props.name}</h3>
        <button onClick={() => this.setState({
count: state.count + 1 })}>
          Cantidad: {state.count}
        </button>
      </div>
    );
  }
}
```